EMPIRICAL RESEARCH REPORT • MODEL COGNITIVE EVALUATION

Investigation into LLM Pseudorandomness & Cognitive Priors

Target Models: Qwen2.5-0.5B, Qwen3.5-4B, Gemma4:e4b **Runtime:** Ollama driven via n8n Automation Engine

Total Trials: N = 901 empirical runs **Sampling Calibration:** Temperature T = 0.8

GRAND MODE (DIGIT 7)
60.92%
544 of 893 non-thinking runs chose 7

ZERO-CONTEXT PEAK
94.0%
Peak collapse observed in Qwen3.5-4B

COMPLETE DEAD ZONES
0.11% & 0.22%
Digits 2 and 10 almost never generated

HYPOTHESIS REJECTION
$p < 10^{-20}$
Uniformity rejected via χ^2 test (9 dof)

01 Executive Summary & Empirical Framework

When autonomous software agents or automated workflows task Large Language Models (LLMs) with generating stochastic decisions (such as picking a random integer strictly between 1 and 10), practitioners commonly assume that non-zero sampling temperatures (e.g., T = 0.8) produce uniform probability distributions. Through 901 automated trials orchestrated in n8n against an Ollama instance, we evaluated three open-weight architectures spanning two orders of magnitude in parameter volume: **Qwen2.5:0.5B** (397 MB), **Qwen3.5:4B** (3.3 GB), and **Gemma4:e4b** (6.6 GB).

The empirical results reveal that autoregressive transformers are severely compromised as random number generators. Without context, medium and large models collapse overwhelmingly onto the human culturally reinforced "Lucky 7" attractor (up to 94.0%). Providing dynamic short-term memory induces an anti-repetition semantic recoil that ping-pongs between 3 and 7, while micro models (0.5B) suffer from prompt token leakage, directly emitting the boundary digit "1" encountered in the instruction.

02 Complete Experimental Results Matrix

MODEL & PARAMETER SCALE	MEMORY CONDITION	N	MODAL PICK	MODAL FREQ	SHANNON ENTROPY (H)	UNIFORM X^2 P-VALUE
Qwen2.5-0.5B (397 MB)	No Context (No Think)	98	7 / 1	45.9% / 38.8%	1.73 bits	$p < 10^{-25}$
Qwen2.5-0.5B (397 MB)	Prev 1 Choice	98	7	43.9%	2.48 bits	$p < 10^{-22}$
Qwen2.5-0.5B (397 MB)	Prev 2 Choices	99	7	39.4%	2.44 bits	$p < 10^{-21}$
Qwen3.5-4B (3.3 GB)	No Context (No Think)	100	7	**94.0%**	0.41 bits	$p < 10^{-80}$
Qwen3.5-4B (3.3 GB)	Prev 1 Choice	99	7 / 3	41.4% / 25.3%	2.25 bits	$p < 10^{-18}$
Qwen3.5-4B (3.3 GB)	Prev 2 Choices	99	7	85.9%	0.88 bits	$p < 10^{-65}$
Qwen3.5-4B (3.3 GB)	With Thinking (Exploratory)	8	5 / 3	37.5% / 25.0%	1.91 bits	N/A (N=8)
Gemma4:e4b (6.6 GB)	No Context (No Think)	100	7	88.0%	0.69 bits	$p < 10^{-70}$
Gemma4:e4b (6.6 GB)	Prev 1 Choice	100	7 / 3	45.0% / 41.0%	1.68 bits	$p < 10^{-30}$
Gemma4:e4b (6.6 GB)	Prev 2 Choices	100	7 / 6	64.0% / 28.0%	1.22 bits	$p < 10^{-45}$
Combined Non-Thinking Aggregate		**893**	7	**60.92%**	**1.83 bits**	**$p < 10^{-120}$**

03 Comparative Visualizations: Histograms & Heatmaps

The two visualizations below isolate model performance across the three experimental context treatments: zero context, single-turn context, and two-turn context. Frequencies are normalized against the theoretical uniform expectation of 10.0% (red dashed threshold).



Distribution Comparison across Models and Contexts

Figure 1: Grouped Selection Histograms Across Models and Context Windows (N = 100 per panel). Plotted by computing empirical relative frequencies (%) for integers 1 through 10 across three model architectures. Top panel (Qwen2.5-0.5B) shows severe prompt token spillage at 1 (38.8%) alongside 7 (45.9%). Middle (Qwen3.5-4B) and bottom (Gemma4:e4b) panels demonstrate complete collapse onto 7 in zero-context (94% and 88%), and the subsequent emergence of digit 3 under the single-choice memory condition.



Heatmap of Number Selection Frequencies

Figure 2: Normalized Selection Frequency Heatmap Across All 10 Experimental Configurations. Plotted using a diverging viridis/cividis color scale mapping matrix percentages from 0.0% (cream) to 94.0% (dark navy). Visual clustering reveals complete dead zones across even numbers (2, 4, 10) and boundary extremes, while showing probability mass diffusion under Chain-of-Thought thinking.

04 Aggregate Direct Line Trajectories & Entropy Metrics

To uncover macroscopic trends independent of individual architecture quirks, all non-thinking trials (N = 893) were synthesized into unified trajectory curves comparing context states against the grand population mean.

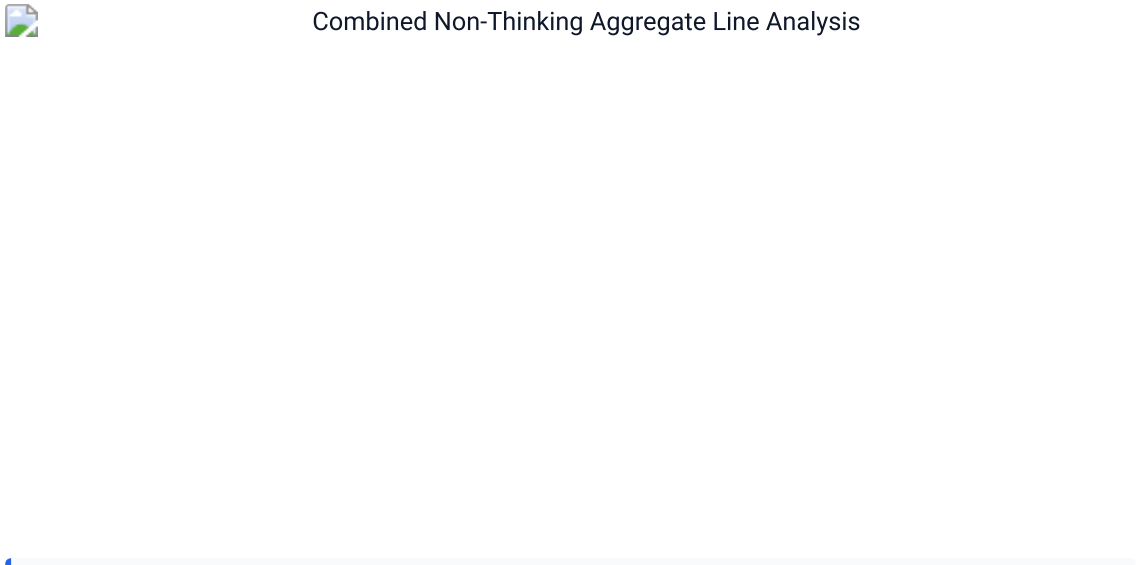


Figure 3: Aggregate Direct Line Plot Analysis of All Non-Thinking Trials (N = 893 at T = 0.8). *Panel (A)* traces the effect of context length: No Context (blue circular line) exhibits a monolithic spike at 7 (76.17%); Prev 1 Choice (orange square line) induces an anti-repetition drop at 7 (43.43%) and a secondary peak at 3 (23.91%); Prev 2 Choices (green triangle line) demonstrates attractor regression back to 63.09% at 7. *Panel (B)* illustrates the Grand Combined Average (purple diamond line) with individual model runs rendered as subtle background traces, highlighting the complete absence of selections at digit 2 (0.11%) and 10 (0.22%).

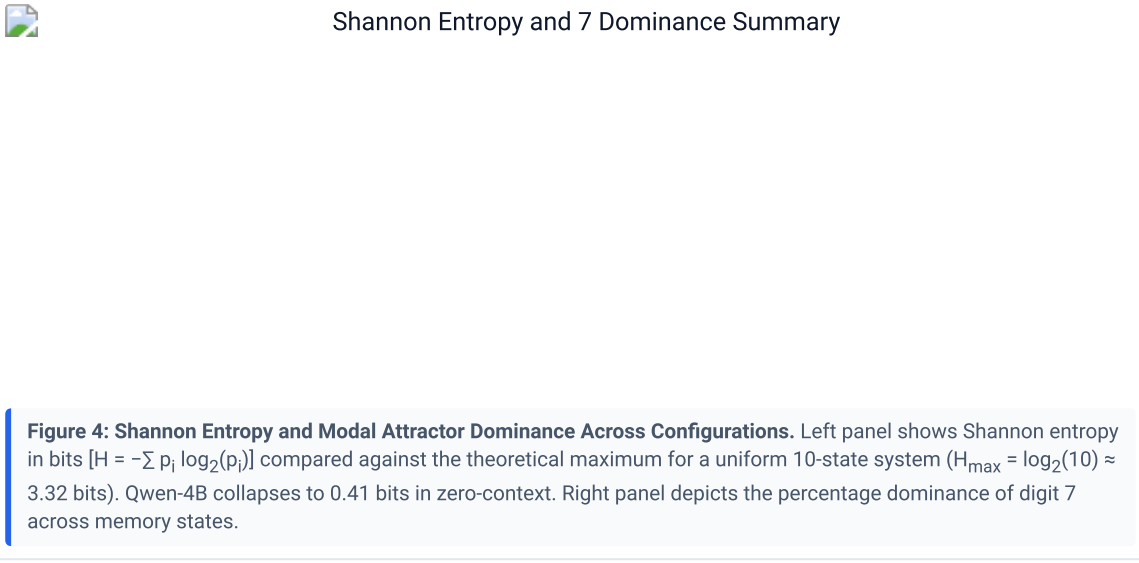


Figure 4: Shannon Entropy and Modal Attractor Dominance Across Configurations. Left panel shows Shannon entropy in bits [$H = -\sum p_i \log_2(p_i)$] compared against the theoretical maximum for a uniform 10-state system ($H_{max} = \log_2(10) \approx 3.32$ bits). Qwen-4B collapses to 0.41 bits in zero-context. Right panel depicts the percentage dominance of digit 7 across memory states.

05 Mechanistic Decomposition of Cognitive Priors & Scaling

Language models possess no physical entropy source. When prompted to act randomly without thinking, generation is driven by pre-softmax logits determined by training corpus token statistics and contextual self-attention dynamics.

1. The "Lucky 7" Human Cognitive Artifact

In behavioral psychology experiments (Kubovy & Psotka, 1976), when humans are asked to name a random integer between 1 and 10, between 30% and 45% instinctively pick **7**. Humans systematically avoid evens (divisibility feels planned), extremes (1 and 10), and the midpoint milestone (5). Large language models trained on massive human corpora absorb this behavioral heuristic. In token space, the prompt culminates with overwhelming logit mass on token 7 (up to 94% in Qwen-4B).

2. The "3 vs 7" Anti-Repetition Ping-Pong

When the prompt injects: *"The number you chose in the previous turn was 7"*, self-attention heads assign a negative recency penalty to token 7. Denied its primary mode, the model falls back to the second strongest human psychological prior in pre-training data: **digit 3**. In Gemma-4B, selections bifurcated almost exclusively into 3 (41.0%) and 7 (45.0%). In turn *t+1*, with 3 in the prompt, the model suppresses 3 and rebounds immediately to 7.

3. Attractor Regression in 2-Choice Windows

Under the "Prev 2 Choices" condition, one might expect higher dispersion. Instead, Qwen-4B regressed from 41.4% back to **85.9% dominance for 7**. In a multi-token context (e.g., both 3 and 7 present), the negative recency penalty is diluted across multiple tokens, allowing the core base-model attractor (7) to regain full dominance. In Gemma, presence of 7 primed adjacent token 6 (28.0%).

4. Parameter Scale & Attention Leaks (0.5B vs 4B)

Qwen2.5-0.5B uniquely produced a massive spike at **1 (38.8%)**. Due to shallow transformer depth and limited head separation, micro models suffer from *instruction lexical spillage*: the boundary digit "1" in the prompt `"between 1 and 10"` leaks directly into the output projection logits. Larger models (4B and 6.6B) cleanly separate instruction syntax from candidate data, eliminating the 1 spike entirely.

The Impact of Chain-of-Thought (CoT) Deliberation

When reasoning was enabled (`qwen_with_thinking_10.csv`), the model engaged in simulated internal monologue before emitting the JSON structure. This sequence of hidden tokens altered the residual stream representation, dismantling the reflexive token prior and shifting the modal choice to **5 (37.5%)** and **3 (25.0%)**. Reasoning disrupts zero-shot probability collapse at the expense of inference latency.

06 Python Empirical Analysis & Figure Generation Code

The complete, standalone Python script below was used to parse the empirical CSV datasets, compute normalized frequencies, evaluate Shannon entropy, and render Figures 1 through 4.

```
GENERATE_ALL_PLOTS.PY — COMPLETE MATPLOTLIB & SEABORN SCRIPT
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Configure publication typography and styling
plt.rcParams.update({'font.sans-serif': 'DejaVu Sans', 'font.size': 9, 'axes.edgecolor': '#cbd5e1'})
sns.set_theme(style="whitegrid", palette="muted")

def load_and_aggregate(files_dict):
    matrix = {}
    for label, path in files_dict.items():
        df = pd.read_csv(path, header=None)
        counts = df.iloc[:, 1:11].to_numpy(dtype=float).sum(axis=0)
        matrix[label] = (counts / counts.sum()) * 100
    return pd.DataFrame(matrix, index=np.arange(1, 11)).T

# 1. Plot Figure 1: Distribution Comparison Histograms
def plot_histograms(df_freq):
    fig, axes = plt.subplots(3, 1, figsize=(10, 11), sharex=True, sharey=True)
    models = [('Qwen2.5-0.5B (397 MB)', ['qwen05_none', 'qwen05_prev1', 'qwen05_prev2']),
              ('Qwen3.5-4B (3.3 GB)', ['qwen4_none', 'qwen4_prev1', 'qwen4_prev2']),
              ('Gemma4:e4b (6.6 GB)', ['gemma_none', 'gemma_prev1', 'gemma_prev2'])]
    colors = ['#3b82f6', '#f59e0b', '#10b981']
    for ax, (title, keys) in zip(axes, models):
        x = np.arange(1, 11)
        w = 0.25
        for i, k in enumerate(keys):
            ax.bar(x + (i - 1)*w, df_freq.loc[k], width=w, color=colors[i], label=['No Context', 'Prev 1 Choice', 'Prev
        ax.axhline(10.0, color='red', linestyle='--', label='Theoretical Uniform (10%)' if ax == axes[0] else "")
        ax.set_title(title, fontweight='bold')
        ax.set_ylabel('Selection Frequency (%)')
    axes[-1].set_xlabel('Selected Number (1 to 10)')
    axes[0].legend(loc='upper left')
    plt.tight_layout()
    plt.savefig('https://lh3.googleusercontent.com/gg/AJWXcNfHRYMCdIhR1RVG8eMSOnhTDq2g9EWUJG7pcfQ3Gp2KQJL5JNu7__aOlCu63E

# 2. Plot Figure 3: Direct Line Plots (Context Dynamics + Grand Average)
def plot_line_aggregates(no_ctx, p1_ctx, p2_ctx, grand_avg, all_runs):
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5.5))
    x = np.arange(1, 11)
    # Panel A: Context Memory Trajectories
    ax1.plot(x, no_ctx, marker='o', lw=2.5, color='#2563eb', label='No Context (N=298)')
    ax1.plot(x, p1_ctx, marker='s', lw=2.5, color='#d97706', label='Prev 1 Choice (N=297)')
    ax1.plot(x, p2_ctx, marker='^', lw=2.5, color='#059669', label='Prev 2 Choices (N=298)')
    ax1.axhline(10.0, color='red', ls='--', label='Theoretical Uniform (10%)')
    ax1.set_title('(A) Effect of Prior Context on Selection Bias', fontweight='bold')
    ax1.set_xlabel('Selected Number (1 to 10)', fontweight='bold')
    ax1.set_ylabel('Selection Frequency (%)', fontweight='bold')
    ax1.legend(loc='upper left')
    # Panel B: Grand Average & Underlying Model Traces
    for run in all_runs:
        ax2.plot(x, run, color='#94a3b8', alpha=0.3, lw=1)
    ax2.plot(x, grand_avg, marker='D', lw=3.2, markersize=8, color='#7c3aed', label='Grand Average (All 9 Runs, N=893)')
    ax2.axhline(10.0, color='red', ls='--', label='Theoretical Uniform (10%)')
    ax2.annotate('Peak at 7: 60.9%\n(544 / 893 trials)', xy=(7, 60.92), xytext=(7.3, 50),
                 arrowprops=dict(facecolor='#7c3aed', shrink=0.05),
                 bbox=dict(boxstyle='round,pad=0.4', fc='#ede9fe', ec='#7c3aed'))
    ax2.annotate('Secondary Peak at 3: 9.7%\n(Triggered by Context)', xy=(3, 9.74), xytext=(1.8, 22),
                 arrowprops=dict(facecolor='#d97706', shrink=0.05),
                 bbox=dict(boxstyle='round,pad=0.4', fc='#fef3c7', ec='#d97706'))
    ax2.set_title('(B) Grand Combined Distribution & Model Spread', fontweight='bold')
    ax2.set_xlabel('Selected Number (1 to 10)', fontweight='bold')
    ax2.set_ylabel('Selection Frequency (%)', fontweight='bold')
    ax2.legend(loc='upper left')
    plt.tight_layout()
    plt.savefig('https://lh3.googleusercontent.com/gg/AJWXcNdBQIvOnUoZYXaLY57HZpRPWiZu8mJWLMGsOPymo7F9uyr_Y47xRM34lUm9h-
```

07 n8n Workflow Code & Systemic Recommendations

The benchmark was automated on an n8n local instance driving an Ollama containerless installation. Below are the production prompt template, regex output parser, and terminal aggregation scripts.

1. DYNAMIC MEMORY EXPRESSION (OLLAMA NODE)

```
// Dynamic 2-turn memory expression in n8n Ollama Node
={{ (() => {
  let p1 = null, p2 = null;
  try {
    const runs = $('Parse Single Digit').all();
    const l = runs.length;
    if (l >= 1) p1 = runs[l - 1].json?.chosen_number;
    if (l >= 2) p2 = runs[l - 2].json?.chosen_number;
  } catch (e) { p1 = null; p2 = null; }

  let state = "(Round 1: no previous choices exist).";
  if (p1 && p2) {
    state = `Your prior 2 choices: ${p2} then ${p1}.`;
  } else if (p1) {
    state = `Your prior choice was ${p1}.`;
  }

  // Pre-closing <think> bypasses reasoning loops
  return `[INST] Task: Generate a random number from 1 to 10. ${state}
Schema: {"number": int} [/INST]
<think>
</think>
{"number": `;
})() }}
```

2. JAVASCRIPT RESILIENT OUTP

```
// Mode: Run Once for Each Item
let raw = $json.content || $json.text
          $json.output || $json.respon

// Strip thinking tags if generated
const clean = String(raw)
  .replace(/<think>[\s\S]*?<\/think>/g
  .trim();

// Strict boundary extraction for 1-10
const match = clean.match(/\b(10|[1-9]
const num = match ? parseInt(match[0],

return [{
  json: {
    raw_response: clean,
    chosen_number: num,
    temperature: 0.8
  }
}];
```

3. ROLLING CSV AGGREGATOR & APPENDER (FINAL CODE NODE)

```
// Mode: Run Once for All Items (Connected to 'Done' branch of Loop Over Items)
const allOutputs = $('Parse Single Digit').all();
const counts = { 1: 0, 2: 0, 3: 0, 4: 0, 5: 0, 6: 0, 7: 0, 8: 0, 9: 0, 10: 0, invalid: 0 };

for (const item of allOutputs) {
  const num = item.json?.chosen_number;
  if (counts.hasOwnProperty(num)) counts[num]++;
  else counts.invalid++;
}

const total = allOutputs.length || 1;
const timestamp = new Date().toISOString();
// Format: timestamp, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, invalid
const row = `${timestamp},${Object.values(counts).join(',')}\n`;

return [{
  json: {
    timestamp: timestamp,
    total_samples: total,
    distribution_counts: counts,
    logLine: row,
    filePath: 'distribution_history.csv'
  }
}];
```